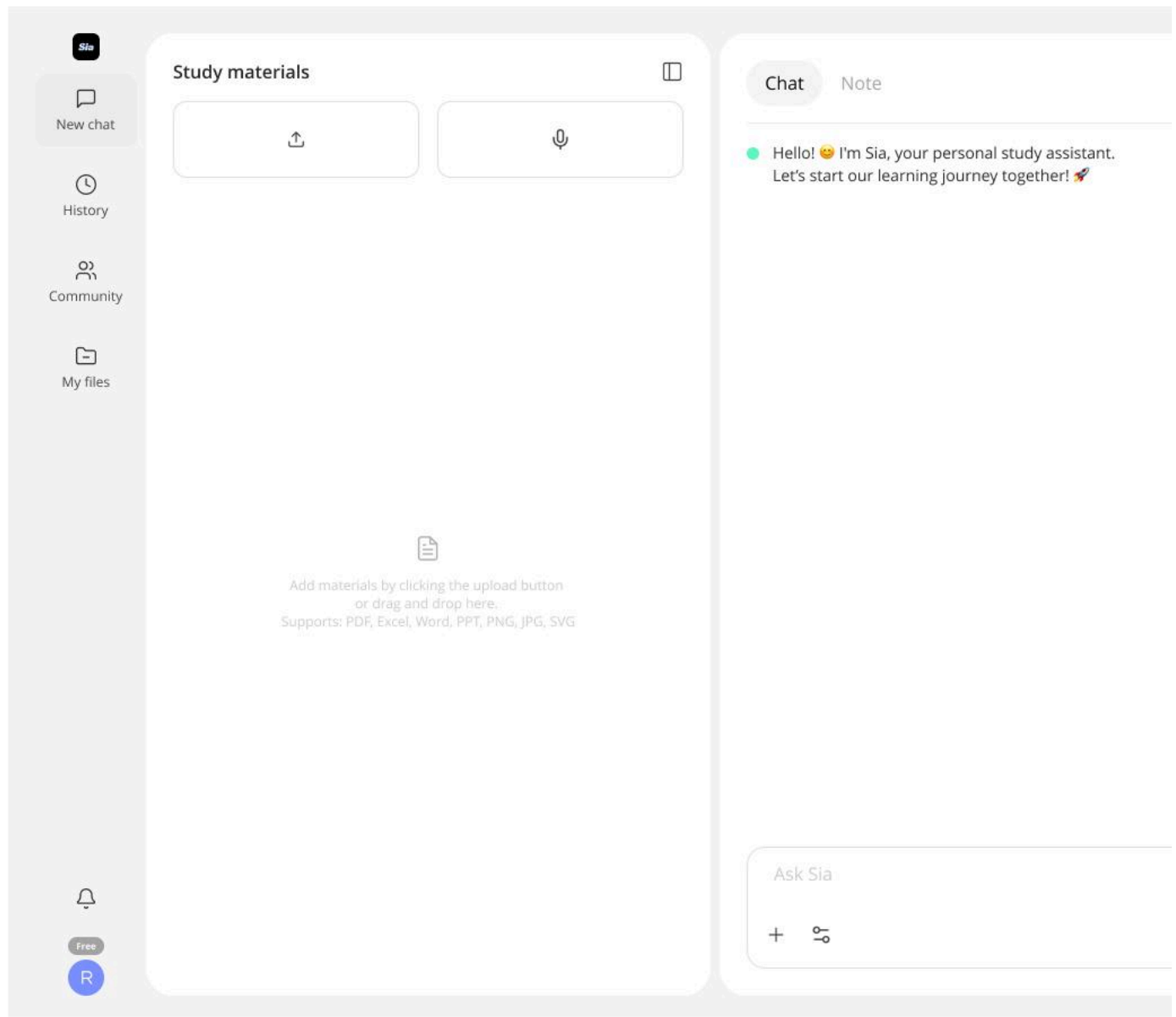




What the application does

User creates a note:

Title: Random Forest **Subject:** Machine Learning **Content:** Random Forest is an ensemble learning algorithm...

Then the user can:

-  Create note
-  View notes

-  Edit note
-  Delete note
-  Ask AI about the note
-  Generate summary
-  Generate quiz questions
-  Generate tags

Technology responsibilities

Part	Technology	Purpose
Frontend	Next.js	UI/dashboard
Backend	FastAPI	REST API
Database	PostgreSQL/Supabase	Store notes
AI	LangChain	LLM operations
Workflow	LangGraph	AI processing workflow
LLM	OpenAI/Ollama	Generate AI responses
Styling	Tailwind CSS	UI
Validation	Pydantic	API schemas

1. CRUD operations

This is the main part of the project.

Create

POST /api/notes

```
{
  "title": "Random Forest",
  "subject": "Machine Learning",
  "content": "Random Forest is an ensemble learning algorithm..."
}
```

Read

GET /api/notes

or

GET /api/notes/{note_id}

Update

PUT /api/notes/{note_id}

Delete

```
DELETE /api/notes/{note_id}
```

This gives you proper **full-stack CRUD experience**.

2. Add LangChain

Once CRUD works, add AI.

For example:

```
POST /api/notes/{note_id}/summarize
```

FastAPI gets the note:

```
Database
↓
FastAPI
↓
LangChain
↓
LLM
↓
Summary
↓
Next.js
```

The user sees:

AI Summary

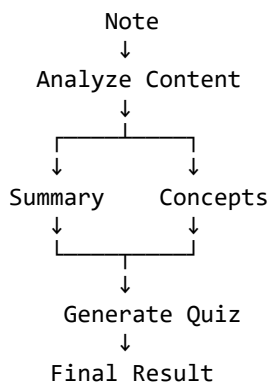
Random Forest combines multiple decision trees to improve prediction accuracy and reduce overfitting.

3. Add LangGraph

Don't use LangGraph for simple CRUD.

Use it specifically for an **AI workflow**.

For example:



Your LangGraph could have:

```
analyze_note
  ↓
generate_summary
  ↓
extract_concepts
  ↓
generate_quiz
  ↓
finalize
```

This is a very good small project for learning LangGraph because the workflow is understandable.

4. Frontend pages

Keep the Next.js application simple.

```
/
├── Dashboard
├── /notes
│   └── All Notes
├── /notes/new
│   └── Create Note
├── /notes/[id]
│   └── View Note
└── /notes/[id]/edit
    └── Edit Note
```

Dashboard:

StudyNotes		+ New Note
Search notes...		
Random Forest ML	Neural Nets Deep Learning	
View Edit	View Edit	

Note page:

Random Forest
Machine Learning

Random Forest is an ensemble
learning algorithm...

[ Edit] [ Delete]

AI Tools

- [Summarize]
 - [Generate Quiz]
 - [Extract Concepts]
 - [Ask AI]
-

5. Database

You only need one main table initially.

notes

```
notes
-----
id
title
subject
content
summary
created_at
updated_at
```

Later you can add:

```
users
subjects
quiz_questions
ai_generations
```

But **don't start with these**.

Start with one table.

6. FastAPI structure

A clean backend could look like:

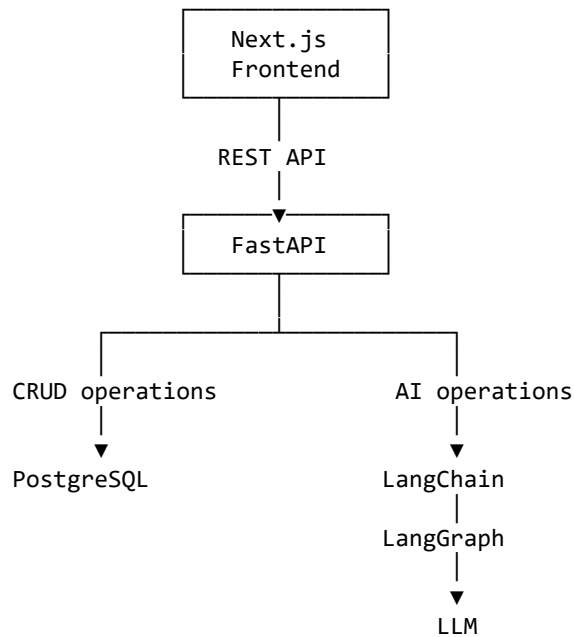
```
backend/
├── app/
│   ├── main.py
│   ├── api/
│   │   └── notes.py
│   ├── models/
│   │   └── note.py
│   ├── schemas/
│   │   └── note.py
│   ├── services/
│   │   ├── note_service.py
│   │   └── ai_service.py
│   ├── db/
│   │   ├── database.py
│   │   └── crud.py
│   └── ai/
│       ├── chains.py
│       └── graph.py
└── requirements.txt
```

7. Next.js structure

```
frontend/  
├── app/  
│   ├── page.tsx  
│   ├── notes/  
│   │   ├── page.tsx  
│   │   ├── new/  
│   │   │   └── page.tsx  
│   │   ├── [id]/  
│   │   │   ├── page.tsx  
│   │   │   └── edit/  
│   │   │       └── page.tsx  
│   └── components/  
│       ├── NoteCard.tsx  
│       ├── NoteForm.tsx  
│       ├── NoteList.tsx  
│       └── AIResult.tsx  
├── lib/  
│   └── api.ts  
└── types/  
    └── note.ts
```

8. API architecture

The overall architecture is:



9. Suggested development order

Don't build everything simultaneously.

Phase 1 — Next.js

Build:

- Dashboard
- Note list
- Create form
- Edit form
- Delete button
- Note detail page

Use dummy data initially.

Phase 2 — FastAPI

Create:

```
POST /notes
GET /notes
GET /notes/{id}
PUT /notes/{id}
DELETE /notes/{id}
```

Test everything using Swagger.

Phase 3 — Database

Connect PostgreSQL/Supabase.

Replace dummy data with real database operations.

Phase 4 — Connect Next.js → FastAPI

Now:

```
Next.js
  ↓
FastAPI
  ↓
PostgreSQL
```

Your CRUD application is complete.

Phase 5 — LangChain

Add:

```
POST /notes/{id}/summarize
POST /notes/{id}/quiz
POST /notes/{id}/concepts
```

Phase 6 — LangGraph

Combine the AI operations into a workflow:

```
Note
  ↓
Analyze
  ↓
Summary
  ↓
Concepts
  ↓
Quiz
  ↓
Result
```

Why I recommend this project for you

It's small enough that you can actually **finish it**, but it teaches almost everything you want:

Next.js → pages, components, forms, API calls, state

FastAPI → routers, endpoints, Pydantic, services, error handling

PostgreSQL → models, relationships, CRUD

LangChain → prompts, chains, structured output, LLM calls

LangGraph → state, nodes, edges, workflows

Full-stack architecture → frontend → backend → database → AI → frontend

And you can later turn it into a stronger portfolio project by adding **RAG, authentication, streaming AI responses, file/PDF uploads, and chat with notes**.

Even simpler alternatives

If you don't like the study-notes idea, these are also good:

1. **AI Expense Tracker** — CRUD expenses + AI spending analysis
2. **AI Resume Manager** — CRUD resumes + AI improvement
3. **AI Book Library** — CRUD books + AI summaries/recommendations
4. **AI Task Manager** — CRUD tasks + AI prioritization
5. **AI Job Application Tracker** — CRUD applications + AI job-description analysis

For learning the stack, I'd pick #1 or the Study Notes Manager.